READING YOUR MIND

EEG signal classification to determine your brain activity before the actual body function using MATLAB.

Done by:

NAME: Abdallah Khairy Werby
ID: 80708

Supervised by: Dr. Eslam Abd El-Azeem

Table of contents

Table of figures

Abstract

The objective of this project is to classify electroencephalography (EEG) data from four different users using machine learning techniques. The EEG data was collected with 14 channels with two features (STD and Mean), and consists of brain waves in the delta, theta, alpha, and beta frequency bands. The data was first preprocessed to extract 1-second segments of data for each of the four frequency bands. The data was then scaled using normalize function, and labels indicating the user were added. The resulting dataset was shuffled and split into training and test sets using a holdout cross-validation technique, with 50% of the data reserved for testing. A linear discriminant analysis classifier was trained on the training set and used to predict the labels of the test set. The accuracy of the classifier was then calculated as the percentage of correctly classified samples. The results showed that the classifier was able to accurately classify the EEG data from the two users, with an average accuracy of 83.3%. This demonstrates the potential of machine learning techniques for accurately classifying EEG data and potentially identifying patterns in brain activity that can be used for various applications such as diagnosis and monitoring of neurological conditions.

Introduction

In this project, we aim to classify brain waves recorded from electroencephalography (EEG) signals into different categories, such as delta, theta, alpha, and beta waves, using machine learning techniques. EEG is a non-invasive technique that measures the electrical activity of the brain by placing electrodes on the scalp. Different types of brain waves are associated with different mental states, such as alertness, relaxation, and sleep, and their analysis can provide insights into brain function and disorders.

To classify the brain waves, we first collected and preprocessed EEG data from four individuals, referred to as User A, User B, User C and User D. The data were recorded at a sampling rate of 160 Hz and were segmented into one-second epochs. We then extracted the delta, theta, alpha, and beta waves from the data using appropriate frequency bands and concatenated the waves from different channels into a single feature vector.

Next, we applied normalization to scale the data and used a holdout cross-validation technique to split the data into training and test sets. We trained a linear discriminant analysis (LDA) classifier on the training data and evaluated its performance on the test data. We also used k-fold cross-validation to assess the robustness of the classifier and to estimate the generalization error, and other types of classifiers.

Finally, we discussed the results of the classification and the implications of the findings for brain wave analysis and applications.

Methodology

1. Load the EEG data from two users into MATLAB. The data is stored in a table format and needs to be converted to an array for further processing.
2. Preprocess the data by removing any rows with invalid values.
3. Extract the delta, theta, alpha, and beta waves from the data. This is done by selecting specific columns in the data array that correspond to these wave types.
4. Scale the data using the normalize function to ensure that all data is on the same scale.
5. Add labels to the data indicating whether the data is from user A or user B.
6. Shuffle the data and labels to ensure that they are randomly distributed.
7. Split the data and labels into training and test sets using a holdout cross-validation technique.
8. Train a classifier on the training data using the classify function and a diagonal linear kernel.
9. Test the classifier on the test data and calculate the accuracy by comparing the predicted labels to the actual labels.

Data

The data for this project consists of electroencephalography (EEG) recordings from 28 channels, collected from four individuals (referred to as User A, User B, User C and User D) while they performed a task. The task involves visualizing a series of images, and the EEG recordings are taken during the viewing of each image. The data is organized in a matrix format, with each row representing a timepoint (in samples) and each column representing a specific channel or feature (such as mean or standard deviation of the EEG signal in a specific frequency range). The first column of the matrix contains labels indicating the type of image being viewed at each timepoint.

The data has been pre-processed to extract features related to four frequency bands: delta (0.5-4 Hz), theta (4-8 Hz), alpha (8-12 Hz), and beta (12-30 Hz). These features include both the mean and standard deviation of the EEG signal in each band, for each channel. The goal of the project is to use this data to build a classifier that can accurately predict the type of image being viewed based on the EEG recordings.

Steps

Step 1: Data preprocessing

Firstly, We load the data, and then the data given appears to be in Table format, so in order to convert it to an array I used "*Table2Array"* function.

```
load ('C:\subs\Users_Data.mat');


% Converting table to array:
Array_a = table2array(User_a);
```

Figure 1- Loading data and converting the table to array

And then the classification given in the data appears to be in 3 classes (0,1,2), that represents three states:

1- looking at Right arrow image. ⟶
2- looking at Left arrow image. ⟵
3- Holding still. ●

But the third state is unnecessary for our classification, so we have to delete all rows with that third class (2)

```
% ------------------------------------------------------------------------------------------
% Removing the third class (2):

mat_aaa = (Array_a(:,1) > 1);


% Check if there are any rows to remove
if any(mat_aaa)
    % Remove the rows
    Matrix_a = User_d(~mat_aaa,:);
end

Matrix_a = table2array(Matrix_a);
```

Figure 2- Removing class (2) rows

In order to do that we created a function that checks the first column of the arrays Array_a and Array_b and identifying any rows that have a value greater than 1. If any such rows are found, it removes them from the User_a and User_b tables and stores the resulting modified tables in the variables Matrix_a and Matrix_b respectively. The tables are then converted to arrays again using the table2array function.

Step 2: Feature extraction (Epoching)

First approach: Manual epoching

Notes:

1- Sampling rate given = 128

2- Four Waves: (Delta, Theta, Alpha, Beta)

3- 14 Channels and 2 features (STD and Mean)

4- STD appears in even numbers after the first column (class)

5- Mean appears in odd numbers after the first column (class)

6-

In my first approach I tried manual epoching for the data, by taking the 128 sampling rate and naming the STD and Mean for every wave starting with delta_1 and until delta_28, as:

- Delta for STD are 14 with their specific index
- Delta for mean are 14 with their specific index
  And transposing the column output horizontally to be a row.

```
% First second For the delta waves:

% STD
delta_1 = Matrix_a(1: 128, 2);
delta_1 = delta_1.';

% Mean
delta_2 = Matrix_a(1: 128, 3);
delta_2 = delta_2.';

% STD
delta_3 = Matrix_a(1: 128, 10);
delta_3 = delta_3.';

% Mean
delta_4 = Matrix_a(1: 128, 11);
delta_4 = delta_4.';

% STD
```

Figure 3- Manual epoching technique

But after doing this very long process, I concluded that:

1- 128 can't be the sampling rate because brain waves change it's class every 160 reading, so that means if I took 128 as sampling rate, the output of the model won't give real answers, and will corrupt the real data.

2- The manual epoching for 1 second for the 4 waves takes about a thousand line!

First approach: For-loop epoching

Notes:

1- New Sampling rate = 160

Now we start epoching the data but this time using a for loop, to do that, we initialize empty arrays for the 4 waves.

```
% Initialize empty array for delta data
delta_data_a = [];
theta_data_a = [];
alpha_data_a = [];
beta_data_a = [];
```

Figure 4- For loop epoching: Initializing arrays

We create a variable combines the 14 channels with their 2 features as "28" called *"num_channels"*.
And then we make an array of the indices for the waves (STDs and Means combined).

```
% Set number of channels (14x2) "2 Features for the 14 channels"
num_channels = 28;

% Set indices of delta columns
delta_col_indices = [2 3 10 11 18 19 26 27 34 35 42 43 50 51 58 59 66 67 74 75 82 83 90 91 98 99 106 107];

% Set indices of theta columns
theta_col_indices = [4 5 12 13 20 21 28 29 36 37 44 45 52 53 60 61 68 69 76 77 84 85 92 93 100 101 108 109];

% Set indices of alpha columns
alpha_col_indices = [6 7 14 15 22 23 30 31 38 39 46 47 54 55 62 63 70 71 78 79 86 87 94 95 102 103 110 111];

% Set indices of beta columns
beta_col_indices = [8 9 16 17 24 25 32 33 40 41 48 49 56 57 64 65 72 73 80 81 88 89 96 97 104 105 112 113];
```

Figure 5- Naming channels and indices for waves

After that we make our for loop:

It extracts the data for delta, theta, alpha, and beta waves from the matrix Matrix_a in 1-second intervals. The for loop that starts with for i = 1:160:size(Matrix_a,1)-159 iterates over the rows of Matrix_a in intervals of 160 rows. This is because the data in Matrix_a is organized in 160-row chunks, with each chunk representing 1 second of data according to our sampling rate we modified. For each iteration of the loop, the code extracts 1 second of data for each of the four wave types (delta, theta, alpha, and beta).
The (minus 159) stops the for loop at (its total rows minus 159) after I collected the needed data.

The inner for loops that start with:
for j = delta_col_indices
for j = theta_col_indices
for j = alpha_col_indices
for j = beta_col_indices
iterate over the columns of Matrix_a containing data for the respective wave type. For each iteration of these inner loops, the code:

1- selects the rows i to i+159 (1 second of data) and the current column j
2- transposes the selected data
3- concatenates it horizontally with the current data in the array delta_data_second, theta_data_second, alpha_data_second, or beta_data_second.
4- Finally, the code concatenates the current second of data vertically to the array delta_data_a, theta_data_a, alpha_data_a, or beta_data_a to add the data for the current second to the array.

```
% Iterate over rows of Matrix_a in 160-sample intervals
for i = 1:160:size(Matrix_a,1)-159
% Extract 1 second of data for delta waves from Matrix_a
delta_data_second = []; % initialize empty array for delta data of current second
for j = delta_col_indices % iterate over columns containing delta data
    delta_data_second = [delta_data_second Matrix_a(i:i+159, j)']; % select rows i to i+159 and column j, transp
end
delta_data_a = [delta_data_a; delta_data_second]; % concatenate vertically to add current second of delta data
```

Figure 6 - The epoching for loop

For-loop output Tables:

As we can see the data tables:

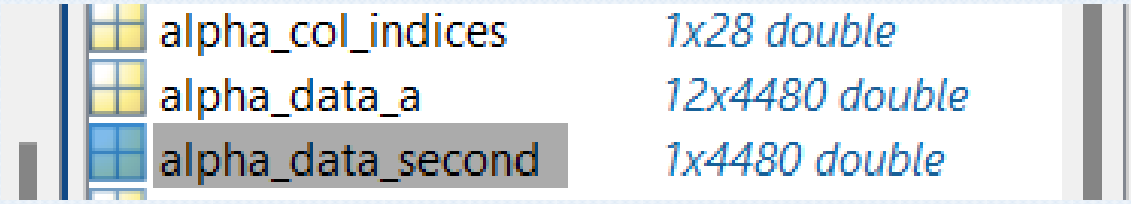
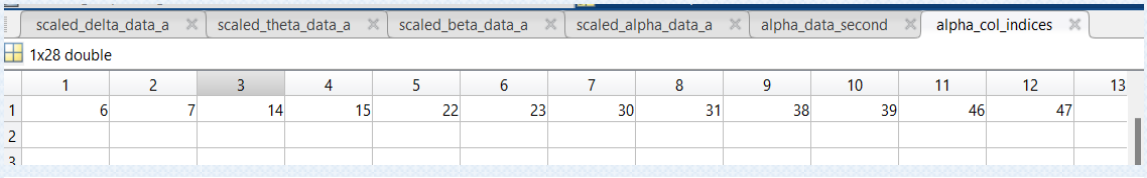


Figure 7- for loop output tables

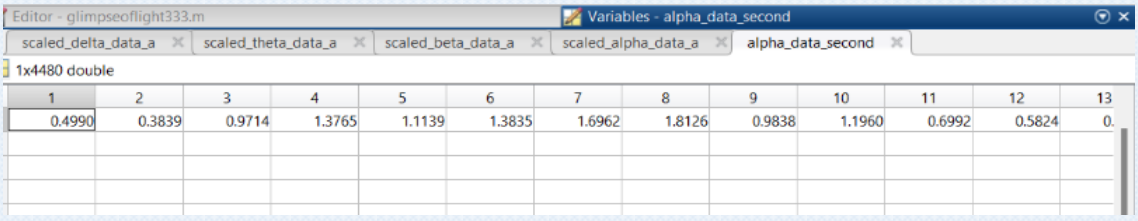
1- The “Indices” is 1x28 Double which is (14 channels with 2 features) but with their right order. For example the alpha is (6,7,14,15,22,23..etc).



	1	2	3	4	5	6	7	8	9	10	11	12	13
1	6	7	14	15	22	23	30	31	38	39	46	47	
2													

Figure 8 - Indices array

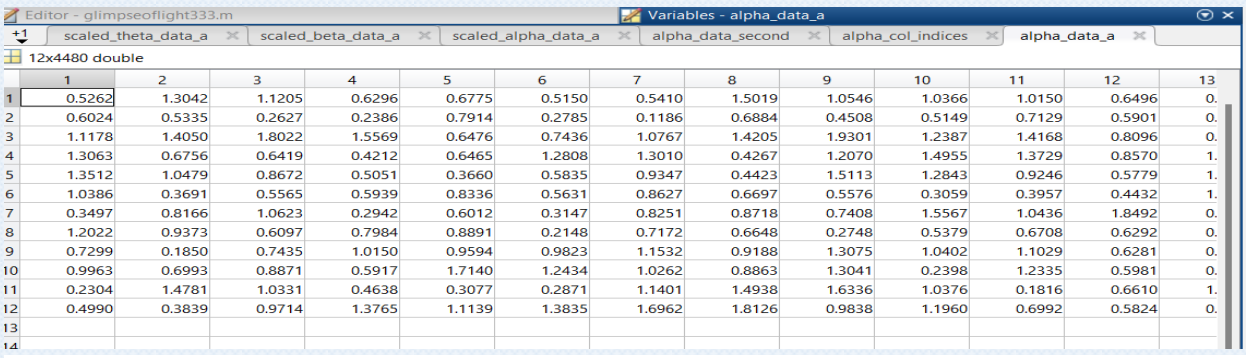
2- The “data_second” the for loop creates is 1x4480, which indicates (1x160) x 28 reading (14 channels with 2 features).



1	2	3	4	5	6	7	8	9	10	11	12	13
0.4990	0.3839	0.9714	1.3765	1.1139	1.3835	1.6962	1.8126	0.9838	1.1960	0.6992	0.5824	0.

Figure 9 - How a wave second is collected in table

3- The “data_a” the for loop creates is 12x4480, which indicates (1x160) x 28 reading (14 channels with 2 features) for the 12 seconds.



	1	2	3	4	5	6	7	8	9	10	11	12	13
1	0.5262	1.3042	1.1205	0.6296	0.6775	0.5150	0.5410	1.5019	1.0546	1.0366	1.0150	0.6496	0.
2	0.6024	0.5335	0.2627	0.2386	0.7914	0.2785	0.1186	0.6884	0.4508	0.5149	0.7129	0.5901	0.
3	1.1178	1.4050	1.8022	1.5569	0.6476	0.7436	1.0767	1.4205	1.9301	1.2387	1.4168	0.8096	0.
4	1.3063	0.6756	0.6419	0.4212	0.6465	1.2808	1.3010	0.4267	1.2070	1.4955	1.3729	0.8570	1.
5	1.3512	1.0479	0.8672	0.5051	0.3660	0.5835	0.9347	0.4423	1.5113	1.2843	0.9246	0.5779	1.
6	1.0386	0.3691	0.5565	0.5939	0.8336	0.5631	0.8627	0.6697	0.5576	0.3059	0.3957	0.4432	1.
7	0.3497	0.8166	1.0623	0.2942	0.6012	0.3147	0.8251	0.8718	0.7408	1.5567	1.0436	1.8492	0.
8	1.2022	0.9373	0.6097	0.7984	0.8891	0.2148	0.7172	0.6648	0.2748	0.5379	0.6708	0.6292	0.
9	0.7299	0.1850	0.7435	1.0150	0.9594	0.9823	1.1532	0.9188	1.3075	1.0402	1.1029	0.6281	0.
10	0.9963	0.6993	0.8871	0.5917	1.7140	1.2434	1.0262	0.8863	1.3041	0.2398	1.2335	0.5981	0.
11	0.2304	1.4781	1.0331	0.4638	0.3077	0.2871	1.1401	1.4938	1.6336	1.0376	0.1816	0.6610	1.
12	0.4990	0.3839	0.9714	1.3765	1.1139	1.3835	1.6962	1.8126	0.9838	1.1960	0.6992	0.5824	0.
13													
14													

Figure 10 - array of 12 Seconds data extracted

Step 3: Scaling the data

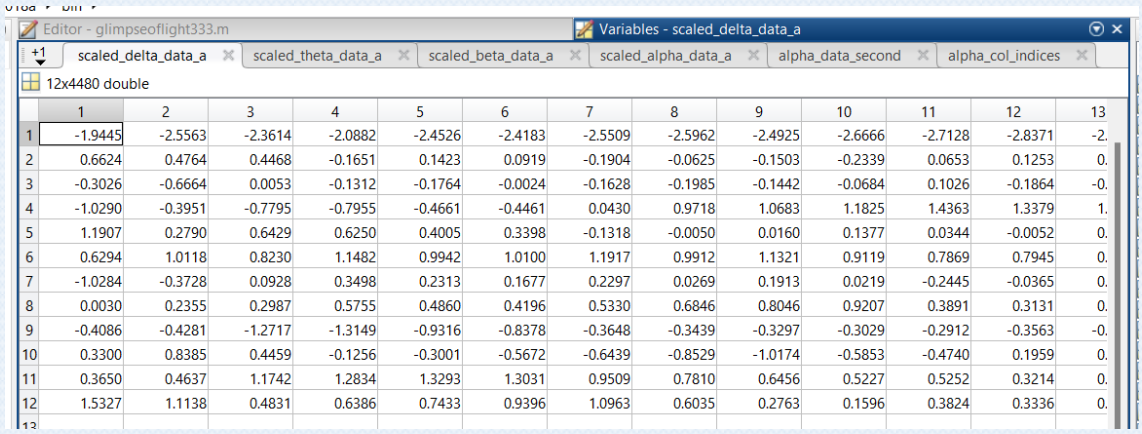
We scale the data we extracted using *normalize* function:

```
% Scale the data using normalize
scaled_delta_data_a = normalize(delta_data_a);
scaled_theta_data_a = normalize(theta_data_a);
scaled_alpha_data_a = normalize(alpha_data_a);
scaled_beta_data_a = normalize(beta_data_a);
```

Figure 11- Scaling the data

Output:

The data in delta for example, after being $x10^3$, now they are scaled to be just like the rest of the data to avoid a biased classification model.



	1	2	3	4	5	6	7	8	9	10	11	12	13
1	-1.9445	-2.5563	-2.3614	-2.0882	-2.4526	-2.4183	-2.5509	-2.5962	-2.4925	-2.6666	-2.7128	-2.8371	-2.
2	0.6624	0.4764	0.4468	-0.1651	0.1423	0.0919	-0.1904	-0.0625	-0.1503	-0.2339	0.0653	0.1253	0.
3	-0.3026	-0.6664	0.0053	-0.1312	-0.1764	-0.0024	-0.1628	-0.1985	-0.1442	-0.0684	0.1026	-0.1864	-0.
4	-1.0290	-0.3951	-0.7795	-0.7955	-0.4661	-0.4461	0.0430	0.9718	1.0683	1.1825	1.4363	1.3379	1.
5	1.1907	0.2790	0.6429	0.6250	0.4005	0.3398	-0.1318	-0.0050	0.0160	0.1377	0.0344	-0.0052	0.
6	0.6294	1.0118	0.8230	1.1482	0.9942	1.0100	1.1917	0.9912	1.1321	0.9119	0.7869	0.7945	0.
7	-1.0284	-0.3728	0.0928	0.3498	0.2313	0.1677	0.2297	0.0269	0.1913	0.0219	-0.2445	-0.0365	0.
8	0.0030	0.2355	0.2987	0.5755	0.4860	0.4196	0.5330	0.6846	0.8046	0.9207	0.3891	0.3131	0.
9	-0.4086	-0.4281	-1.2717	-1.3149	-0.9316	-0.8378	-0.3648	-0.3439	-0.3297	-0.3029	-0.2912	-0.3563	-0.
10	0.3300	0.8385	0.4459	-0.1256	-0.3001	-0.5672	-0.6439	-0.8529	-1.0174	-0.5853	-0.4740	0.1959	0.
11	0.3650	0.4637	1.1742	1.2834	1.3293	1.3031	0.9509	0.7810	0.6456	0.5227	0.5252	0.3214	0.
12	1.5327	1.1138	0.4831	0.6386	0.7433	0.9396	1.0963	0.6035	0.2763	0.1596	0.3824	0.3336	0.

Figure 12 - Scaled data

Step 4: Adding the labels

Firstly, we initialize empty array for Labels, and we concatenate the 4 types of waves that was already scaled horizontally.

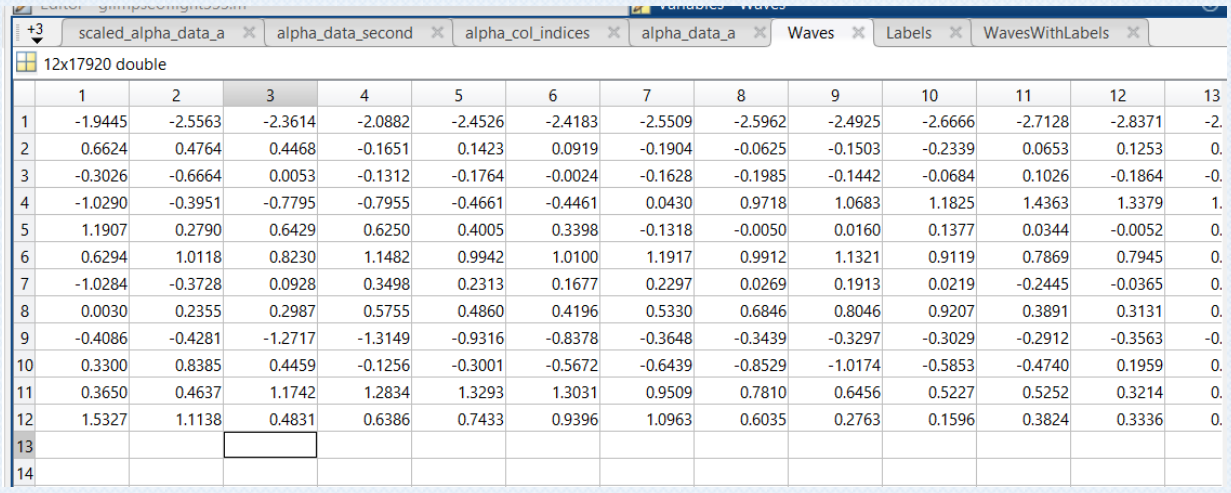
```
% Initialize empty array for labels
Labels = [];

% concatenate the scaled data together of the 4 waves toghether horizontally:
Waves = [scaled_delta_data_a scaled_theta_data_a scaled_alpha_data_a scaled_beta_data_a];
```

Figure 13 - Concatenating scaled data waves

Output:

Waves then will be a table of 12x17920 which is each wave 4480x4 in 12 seconds:



	1	2	3	4	5	6	7	8	9	10	11	12	13
1	-1.9445	-2.5563	-2.3614	-2.0882	-2.4526	-2.4183	-2.5509	-2.5962	-2.4925	-2.6666	-2.7128	-2.8371	-2.
2	0.6624	0.4764	0.4468	-0.1651	0.1423	0.0919	-0.1904	-0.0625	-0.1503	-0.2339	0.0653	0.1253	0.
3	-0.3026	-0.6664	0.0053	-0.1312	-0.1764	-0.0024	-0.1628	-0.1985	-0.1442	-0.0684	0.1026	-0.1864	-0.
4	-1.0290	-0.3951	-0.7795	-0.7955	-0.4661	-0.4461	0.0430	0.9718	1.0683	1.1825	1.4363	1.3379	1.
5	1.1907	0.2790	0.6429	0.6250	0.4005	0.3398	-0.1318	-0.0050	0.0160	0.1377	0.0344	-0.0052	0.
6	0.6294	1.0118	0.8230	1.1482	0.9942	1.0100	1.1917	0.9912	1.1321	0.9119	0.7869	0.7945	0.
7	-1.0284	-0.3728	0.0928	0.3498	0.2313	0.1677	0.2297	0.0269	0.1913	0.0219	-0.2445	-0.0365	0.
8	0.0030	0.2355	0.2987	0.5755	0.4860	0.4196	0.5330	0.6846	0.8046	0.9207	0.3891	0.3131	0.
9	-0.4086	-0.4281	-1.2717	-1.3149	-0.9316	-0.8378	-0.3648	-0.3439	-0.3297	-0.3029	-0.2912	-0.3563	-0.
10	0.3300	0.8385	0.4459	-0.1256	-0.3001	-0.5672	-0.6439	-0.8529	-1.0174	-0.5853	-0.4740	0.1959	0.
11	0.3650	0.4637	1.1742	1.2834	1.3293	1.3031	0.9509	0.7810	0.6456	0.5227	0.5252	0.3214	0.
12	1.5327	1.1138	0.4831	0.6386	0.7433	0.9396	1.0963	0.6035	0.2763	0.1596	0.3824	0.3336	0.
13													
14													

Figure 14 - Concatenated waves in 12 seconds

And then we make a for loop that is iterating over the rows of the array "Matrix_a" in intervals of 160 rows (samples).
For each iteration, it is taking the value of the first column at the current row and adding it to an array called "Labels". After the loop finishes, it is

transposing the "Labels" array and then concatenating it horizontally with the array "Waves" to create a new array called "WavesWithLabels".

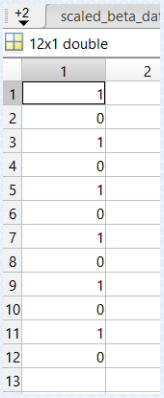
```
% Iterate over rows of Matrix_a in 160-sample intervals
for i = 1:160:size(Matrix_a,1)-159
Labels = [Labels Matrix_a(i, 1)]; % add value of the first column at row i to the array
end
Labels = Labels.';

WavesWithLabels = [ Waves Labels ];
```

Figure 15 - Creating the Labels

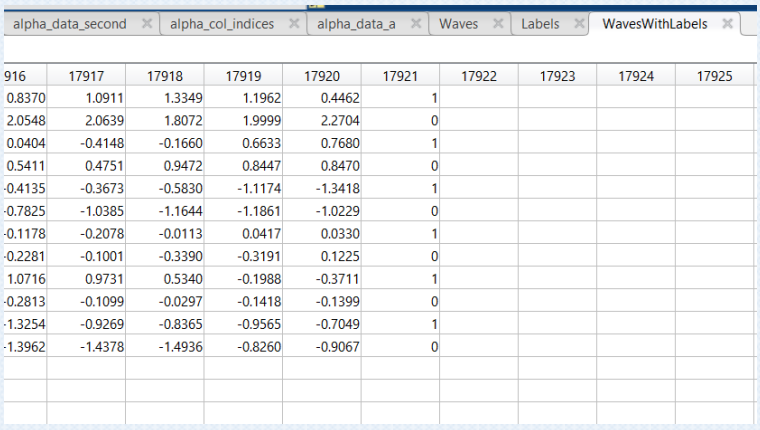
Output:

now waves with labels are 1x19721 after concatenating the labels horizontally



	1	2
1	1	
2	0	
3	1	
4	0	
5	1	
6	0	
7	1	
8	0	
9	1	
10	0	
11	1	
12	0	
13		

Figure 17 - Labels



alpha_data_second | alpha_col_indices | alpha_data_a | Waves | Labels | WavesWithLabels

916	17917	17918	17919	17920	17921	17922	17923	17924	17925
0.8370	1.0911	1.3349	1.1962	0.4462	1				
2.0548	2.0639	1.8072	1.9999	2.2704	0				
0.0404	-0.4148	-0.1660	0.6633	0.7680	1				
0.5411	0.4751	0.9472	0.8447	0.8470	0				
-0.4135	-0.3673	-0.5830	-1.1174	-1.3418	1				
-0.7825	-1.0385	-1.1644	-1.1861	-1.0229	0				
-0.1178	-0.2078	-0.0113	0.0417	0.0330	1				
-0.2281	-0.1001	-0.3390	-0.3191	0.1225	0				
1.0716	0.9731	0.5340	-0.1988	-0.3711	1				
-0.2813	-0.1099	-0.0297	-0.1418	-0.1399	0				
-1.3254	-0.9269	-0.8365	-0.9565	-0.7049	1				
-1.3962	-1.4378	-1.4936	-0.8260	-0.9067	0				

Figure 16 - Waves with labels

Step 5: Randomization

For our next step, we will randomize the WavesWithLabels by shuffling the rows of the WavesWithLabels array using the *randperm* function, which generates a random permutation of the integers from 1 to the number of rows in WavesWithLabels. The resulting shuffled array is then stored in the variable shuffledArray.

Then, The last column of shuffledArray, which contains the labels, is separated into a separate array called Shuffledlabels.

The rest of the array, shuffledArrayNoLabels, is obtained by removing the last column from shuffledArray.

```
% Shuffling the array with labels added to the 2 subs
shuffledArray = WavesWithLabels(randperm(size(WavesWithLabels,1)),:);


% Take the shuffled labels aside
Shuffledlabels = shuffledArray(:,end);


% Take the labels out
shuffledArrayNoLabels = shuffledArray(:,1:end-1);
```

Figure 18 - Randomization and separation of labels and data again

Machine Learning steps

Step 1: Data splitting

To start the machine learning process, we need to split the data:

1- Train
2- Test

By any percentage you want, for my project I used 50% train, 50% test.

```
% Splitting data into 50% train and 50% test
cv = cvpartition(size(shuffledArrayNoLabels,1),'HoldOut',0.5);
idx = cv.test;

dataTrain = shuffledArrayNoLabels(~idx,:);
dataTest  = shuffledArrayNoLabels(idx,:);


% Splitting Labels into 50% train and 50% test
cv = cvpartition(size(Shuffledlabels,1),'HoldOut',0.5);
idx = cv.test;

LabelsTrain = Shuffledlabels(~idx,:);
LabelsTest  = Shuffledlabels(idx,:);
```

Figure 19 - Splitting the data into train and test

Now we are dividing the data and labels into two sets: a training set and a test set. The training set will be used to train a classifier, and the test set will be used to evaluate the performance of the classifier.

The first block of code uses the cvpartition function to create a partition of the data into two sets, with 50% of the data in the test set and 50% in the training set. It then uses this partition to index into the data and divide it into the dataTrain and dataTest sets.

The second block of code does the same thing for the labels, using a separate partition and dividing the labels into LabelsTrain and LabelsTest.

This splitting of the data and labels into training and test sets is a common practice in machine learning, as it allows us to evaluate the performance of a classifier on unseen data and get a better estimate of how well it will generalize to new data.

Second approach

For the Sampling rate of "20"

In the larger amount of seconds. We can actually use cross validation with number of folds, to do that, First of all we set up a stratified k-fold cross-validation procedure for evaluating the performance of a machine learning model. The number of folds is specified as nfolds and is set to 4 in this case. Stratified k-fold cross-validation ensures that the proportion of each class is maintained in each fold, which is important if the classes in the dataset are imbalanced.

The cvpartition function is used to generate the indices for the cross-validation procedure, using the input Labels (which represents the labels or target values for the data) and the number of folds nfolds. The Stratify option is set to true to ensure that the proportion of each class is maintained in each fold.

The accuracy, sensitivity, and specificity variables are initialized as arrays to store the performance measures for each fold. The size of these arrays is set to nfolds so that a separate performance measure can be stored for each fold.

```
% Set number of folds for stratified k-fold cross-validation
nfolds = 4;

% Generate indices for stratified k-fold cross-validation
partition = cvpartition(Labels,'KFold',nfolds, 'Stratify', true);
```

Figure 20 - Second approach of splitting by cross validation and nfolds.

Step 2: Classification

To start the machine learning process we initialize our performance metrics.

```
% Initialize variables to store performance measures
accuracy = zeros(nfolds,1);
sensitivity = zeros(nfolds,1);
specificity = zeros(nfolds,1);
```

This code is part of a loop that performs stratified k-fold cross-validation on a dataset. In each iteration of the loop, the code divides the data into a training set and a test set, and then trains an SVM classifier on the training set. It then uses the trained classifier to make predictions on the test set, and calculates performance measures such as accuracy, sensitivity, and specificity using the function getPerformanceMeasures we are declaring in the last piece of code. These performance measures are stored in the arrays accuracy, sensitivity, and specificity.

The purpose of stratified k-fold cross-validation is to partition the data into folds in a way that preserves the proportions of different classes in the data. For example, if the data has a balanced distribution of labels, each fold should also have a balanced distribution of labels. This helps to ensure that the classifier is trained and tested on a representative sample of the data, and can help to reduce bias and overfitting.

```
% Iterate over folds
for i = 1:nfolds
    % Get training and test sets for current fold
    idx = partition.training(i);
    dataTrain = shuffledArrayNoLabels(idx,:);
    LabelsTrain = Labels(idx);
    idx = partition.test(i);
    dataTest = shuffledArrayNoLabels(idx,:);
    LabelsTest = Labels(idx);

    % Train SVM classifier on training data
    SVMModel = fitcsvm(dataTrain,LabelsTrain);

    % Make predictions on test data
    predictions = predict(SVMModel, dataTest);

    % Calculate performance measures
    [accuracy(i), sensitivity(i), specificity(i)] = getPerformanceMeasures(predictions, LabelsTest);
end
```

Figure 21 - for loop for the iteration of nfolds

Now we calculate the average of the accuracy, sensitivity, and specificity.

```
% Calculate mean and standard deviation of performance measures across folds
meanAccuracy = mean(accuracy)
meanSensitivity = mean(sensitivity);
meanSpecificity = mean(specificity);
```

Figure 22 - Calculating average of accuracy metrics.

Step 2": Normal 160Hz Classification

To do the classification part we are doing 5 classification techniques:

1- DiagLinear
2- Support Vector Machine (SVM)
3- K- Nearest Neighbor (KNN)
4- Logistic regression
5- Naïve Bayes
6- Decision Trees

In this piece of code, we are creating a classifier using the *classify* function with the 'diaglinear' option, which specifies that a linear discriminant analysis classifier with a diagonal covariance matrix should be used. It then compares the predicted labels in the class array with the true labels in the LabelsTest array to count how many predictions are correct. Finally, it calculates the accuracy as the ratio of the number of correct predictions to the total number of predictions, and expresses it as a percentage.

```
% Creating the diaglinear classifier
class = classify(dataTest,dataTrain,LabelsTrain,'diaglinear');


% Measuring accuracy
Correct_Labels = 0 ;

for i=1:length(class)
    if class(i) == LabelsTest(i)
        Correct_Labels = Correct_Labels + 1 ;
    end
end
    diagLinear_Accuracy = (Correct_Labels/length(class))*100
```

Figure 23 - DiagLinear classifier

But as you can see this type of accuracy measuring is considered slow, so we will create our own accuracy metrics function that is written in the end of the code.

My performance metrics function:

The function calculates various performance measures for a given model and its predictions. The input arguments are:

```
% FUNCTION:

function [accuracy, sensitivity, specificity] = getPerformanceMeasures(predictions, labels, modelType)
% Calculate the number of true positive predictions
truePositives = sum((predictions == 1) & (labels == 1));

% Calculate the number of true negative predictions
trueNegatives = sum((predictions == 0) & (labels == 0));

% Calculate the number of false positive predictions
falsePositives = sum((predictions == 1) & (labels == 0));

% Calculate the number of false negative predictions
falseNegatives = sum((predictions == 0) & (labels == 1));

% Calculate the accuracy
accuracy = (truePositives + trueNegatives) / (truePositives + trueNegatives + falsePositives + falseNegatives);

% Calculate the sensitivity
sensitivity = truePositives / (truePositives + falseNegatives);

% Calculate the specificity
specificity = trueNegatives / (trueNegatives + falsePositives);
```

Figure 24 - Performance metrics function

- predictions: a vector of predicted labels, either 0 or 1, for a given model
- labels: a vector of true labels, either 0 or 1, for a given dataset
- modelType: a string indicating the type of model for which the performance measures are being calculated (e.g., "SVM", "KNN", "Naive Bayes", etc.) To avoid overwriting of the metrics.

The output of the function is a tuple of three values:

1- accuracy: a measure of how many predictions made by the model were correct, calculated as the ratio of the number of true positive and true negative predictions to the total number of predictions made
2- sensitivity: a measure of how well the model detects positive instances, calculated as the ratio of the number of true positive predictions to the total number of positive instances

3- specificity: a measure of how well the model detects negative instances, calculated as the ratio of the number of true negative predictions to the total number of negative instances

The function first calculates the number of true positive, true negative, false positive, and false negative predictions made by the model. It then uses these values to calculate the accuracy, sensitivity, and specificity of the model. Finally, it stores the performance measures in variables specific to the given model type (e.g., svmAccuracy, knnSensitivity, etc.). The purpose of this is to allow the user to compare the performance of different models on the same dataset.

```
% Store the performance measures for the current model
switch modelType

    case 'DiagLinear'
        diagAccuracy = accuracy;
        diagSensitivity = sensitivity;
        diagSpecificity = specificity;

    case 'SVM'
        svmAccuracy = accuracy;
        svmSensitivity = sensitivity;
        svmSpecificity = specificity;
    case 'KNN'
        knnAccuracy = accuracy;
        knnSensitivity = sensitivity;
        knnSpecificity = specificity;
    case 'Naive Bayes'
        nbAccuracy = accuracy;
        nbSensitivity = sensitivity;
        nbSpecificity = specificity;
    case 'Decision Tree'
        dtAccuracy = accuracy;
        dtSensitivity = sensitivity;
        dtSpecificity = specificity;
    case 'Random Forest'
        rfAccuracy = accuracy;
        rfSensitivity = sensitivity;
        rfSpecificity = specificity;
        end
```

Figure 25 - Models switching in the function

Second model: KNN

The k-nearest neighbors (KNN) classification model is a type of instance-based learning or lazy learning. It is an intuitive method that classifies a data point based on the class of the majority of the points within a certain distance (a.k.a., the "neighborhood") of that data point.

```
% KNN classification model

% Train KNN classifier
knnModel = fitcknn(dataTrain, LabelsTrain, 'NumNeighbors', 5);

% Make predictions on test data
predictions_KNN = predict(knnModel, dataTest);

% Calculate performance measures
[knnAccuracy, knnSensitivity, knnSpecificity] = getPerformanceMeasures(predictions_KNN, LabelsTest, 'KNN');

% Print the results
fprintf('KNN Model Performance:\n');
fprintf('Accuracy_KNN: %.2f%%\n', knnAccuracy*100);
fprintf('Sensitivity_KNN: %.2f%%\n', knnSensitivity*100);
fprintf('Specificity_KNN: %.2f%%\n', knnSpecificity*100);
```

Figure 26 - KNN model

In this piece of code, the KNN classifier is trained using the *fitcknn* function, which takes the training data (dataTrain) and labels (LabelsTrain) as input arguments. The NumNeighbors parameter specifies the number of nearest neighbors that should be considered when making predictions.

Once the classifier is trained, it can be used to make predictions on new data (dataTest) using the predict function. The performance of the model is then evaluated using the getPerformanceMeasures function, which calculates the accuracy, sensitivity, and specificity of the model on the test data. Finally, the results are printed to the command window.

Third model: Decision Trees

The decision tree is a machine learning model used for classification and regression tasks. It is a flowchart-like tree structure that splits a dataset into smaller and smaller subsets based on different decision rules. The goal is to create a model that predicts the value of a target variable by learning simple decision rules inferred from the data features, but our data is considered small for DT to deal with, along with its iterations and folds.

In our code we are training and evaluating a decision tree classifier using stratified k-fold cross-validation.

First, the number of folds for the cross-validation is set to 2. Then, indices for stratified k-fold cross-validation are generated using the cvpartition function, where Labels is an array of class labels for the data, 'KFold' specifies the type of cross-validation to use, nfolds specifies the number of folds, and 'Stratify' ensures that the class proportions are preserved in each fold.

```
%DT classification model
% Set number of folds for stratified k-fold cross-validation
nfolds = 2;

% Generate indices for stratified k-fold cross-validation
partition = cvpartition(Labels,'KFold',nfolds, 'Stratify', true);

% Iterate over folds
for i = 1:nfolds
% Get training and test sets for current fold
idx = partition.training(i);
dataTrain = shuffledArrayNoLabels(idx,:);
LabelsTrain = Labels(idx);
idx = partition.test(i);
dataTest = shuffledArrayNoLabels(idx,:);
LabelsTest = Labels(idx);

% Train Decision Trees classifier on training data
DecisionTreeModel = fitctree(dataTrain,LabelsTrain);

% Make predictions on test data
predictions_DT = predict(DecisionTreeModel, dataTest);

% Calculate performance measures
[dtaccuracy(i), dtsensitivity(i), dtspecificity(i)] = getPerformanceMeasures(predictions_DT, LabelsTest, 'Dec
end
```

Figure 27 - Decision Trees model

Next, the code iterates over the folds. For each iteration, the training and test sets are obtained using the training and test indices of the partition object, respectively. The training data and labels are used to train a decision tree classifier using the *fitctree* function, and the trained classifier is then used to make predictions on the test data using the predict function. The performance measures (accuracy, sensitivity, and specificity) of the classifier are then calculated on the test data using the getPerformanceMeasures function, which takes as input the predictions, the true labels, and a string identifying the classifier.

```
% Calculate mean and standard deviation of performance measures across folds
meanAccuracy_DT = mean(dtaccuracy);
meanSensitivity_DT = mean(dtsensitivity);
meanSpecificity_DT = mean(dtspecificity);

% Print the results
fprintf('DT Model Performance:\n');
fprintf('Accuracy_DT: %.2f%%\n', meanAccuracy_DT*100);
fprintf('Sensitivity_DT: %.2f%%\n', meanSensitivity_DT*100);
fprintf('Specificity_DT: %.2f%%\n', meanSpecificity_DT*100);
```

Figure 28 - Decision Trees model performance metrics

After all the iterations, the mean of the performance measures across the folds are calculated and printed. The mean values are used as the overall performance measures of the decision tree classifier.

Fourth model: Logistic regression

Logistic regression is a type of classification algorithm that is used to predict a binary outcome (e.g. a yes/no response or a 0/1 label). It is a linear model that is used to model the probability of a certain class or event occurring.

```
%-------------------------------------------------------------------------

% Logistic classification model

% Train logistic classifier on training data
Logisticclassifier = fitclinear(dataTrain,LabelsTrain,'Learner','logistic');

% Make predictions on test data
predictions_LOG = predict(Logisticclassifier, dataTest);

% Calculate performance measures
[dtaccuracy(i), dtsensitivity(i), dtspecificity(i)] = getPerformanceMeasures(predictions_LOG, LabelsTest, 'Logistic');


% Calculate mean and standard deviation of performance measures across folds
meanAccuracy_Logistic = mean(dtaccuracy);
meanSensitivity_Logistic = mean(dtsensitivity);
meanSpecificity_Logistic = mean(dtspecificity);

% Print the results
fprintf('Logistic Model Performance:\n');
fprintf('Accuracy_Logistic: %.2f%%\n', meanAccuracy_Logistic*100);
fprintf('Sensitivity_Logistic: %.2f%%\n', meanSensitivity_Logistic*100);
fprintf('Specificity_Logistic: %.2f%%\n', meanSpecificity_Logistic*100);
%-------------------------------------------------------------------------
```

Figure 29 - Logistic regression model

The first line of code trains a logistic regression classifier on the training data using the *fitclinear* function. The 'Learner' option specifies that a logistic regression model should be used, and the dataTrain and LabelsTrain inputs provide the data and labels that will be used to train the model.

The second line of code uses the predict function to make predictions on the test data using the trained logistic classifier. The predictions are stored in the predictions_LOG array.

The third line of code uses the getPerformanceMeasures function to calculate performance measures such as accuracy, sensitivity, and specificity for the logistic regression model. These performance measures are calculated based on the predicted labels stored in predictions_LOG and the true labels stored in LabelsTest.

The fourth line of code calculates the mean accuracy, sensitivity, and specificity across all folds of the cross-validation procedure. The final block of code prints these mean performance measures to the command window.

Fifth model: SVM

Support Vector Machines (SVMs) are a type of supervised machine learning algorithm that can be used for classification or regression tasks. The goal of an SVM is to find the hyperplane in a high-dimensional space that maximally separates the data points of different classes. The hyperplane is chosen so that it maximally separates the classes and has the largest distance (called the margin) to the nearest data points of any class. The data points that are closest to the hyperplane are called support vectors and have a special role in determining the position and orientation of the hyperplane.

```
% SVM classification model

% Train SVM classifier on training data
SVMModel = fitcsvm(dataTrain,LabelsTrain);

% Make predictions on test data
predictions_SVM = predict(SVMModel, dataTest);

% Calculate performance measures for SVM model
[svmAccuracy, svmSensitivity, svmSpecificity] = getPerformanceMeasures(predictions_SVM, LabelsTest, 'SVM');

% Display performance measures
fprintf('SVM Model Performance:\n');
fprintf('Accuracy_SVM: %.2f%%\n', svmAccuracy*100);
fprintf('Sensitivity_SVM: %.2f%%\n', svmSensitivity*100);
fprintf('Specificity_SVM: %.2f%%\n', svmSpecificity*100);
```

Figure 30 - SVM model

In this piece of code, the SVM classifier is trained using the *fitcsvm* function on the training data (dataTrain) and labels (LabelsTrain). The trained classifier is then used to make predictions on the test data (dataTest) using the predict function. The performance of the classifier is evaluated using the getPerformanceMeasures function, which calculates the accuracy, sensitivity, and specificity of the classifier. The accuracy, sensitivity, and specificity are then printed to the command window.

Sixth model: Naïve bayes

Naive Bayes is a classification algorithm based on the idea of using Bayes' Theorem to make predictions. Bayes' Theorem states that the probability of an event occurring (such as an example belonging to a particular class) can be calculated by using the probabilities of certain related events (such as the presence or absence of certain features in the example).

The "naive" part of the name comes from the assumption that all of the features in the data are independent of one another, which is often not the case in real-world data. Despite this assumption, Naive Bayes classifiers can still be effective in many cases and are often used as a baseline for comparison with more complex models.

In this code, the Naive Bayes model is trained using the *fitcnb* function on the dataTrain and LabelsTrain data. The function returns a trained Naive Bayes model, which is then used to make predictions on the test data using the predict function. The performance of the model is then evaluated using the getPerformanceMeasures function, which calculates accuracy, sensitivity, and specificity measures for the model. Finally, the results are printed to the command window.

```
%--------------------------------------------------------------------------------
% Naive Bayes classification model

% Train the Naive Bayes model
NBModel = fitcnb(dataTrain, LabelsTrain);

% Make predictions on the test data
predictions_NB = predict(NBModel, dataTest);

% Calculate performance measures
[nbAccuracy, nbSensitivity, nbSpecificity] = getPerformanceMeasures(predictions_NB, LabelsTest, 'Naive Bayes');

% Print the results
fprintf('NB Model Performance:\n');
fprintf('Accuracy_NB: %.2f%%\n', nbAccuracy*100);
fprintf('Sensitivity_NB: %.2f%%\n', nbSensitivity*100);
fprintf('Specificity_NB: %.2f%%\n', nbSpecificity*100);


%--------------------------------------------------------------------------------
```

Figure 31 - Naive bayes model

Results

To see the result in a clear form we will create a table that shows how all the models perform.

```
% Results Table

% Create a cell array with the names of the models
models = {'DiagLinear', 'KNN', 'Decision Tree', 'Logistic', 'SVM', 'Naive Bayes'};

% Create a table to store the results
resultsTable = table('Size',[length(models),4], ...
'VariableTypes',{'string','double','double','double'}, ...
'VariableNames',{'Model','Accuracy','Sensitivity','Specificity'});
```

Figure 32 – code for Results table

In this piece of code, we are creating a table to store the results of the performance measures (accuracy, sensitivity, and specificity) of different classification models that have been trained and tested on the data.

The table is created with 'Size',[length(models),4], which specifies that the table should have as many rows as there are models (determined by the length of the 'models' array) and 4 columns. The 'VariableTypes' and 'VariableNames' options specify the data type and names of the columns in the table.

```
% Iterate over the models
for i = 1:length(models)
% Calculate the performance measures for the current model
switch models{i}
case 'DiagLinear'
[accuracy, sensitivity, specificity] = getPerformanceMeasures(predictions_diag, LabelsTest, models{i});
case 'KNN'
[accuracy, sensitivity, specificity] = getPerformanceMeasures(predictions_KNN, LabelsTest, models{i});
case 'Decision Tree'
[accuracy, sensitivity, specificity] = getPerformanceMeasures(predictions_DT, LabelsTest, models{i});
case 'Logistic'
[accuracy, sensitivity, specificity] = getPerformanceMeasures(predictions_LOG, LabelsTest, models{i});
case 'SVM'
[accuracy, sensitivity, specificity] = getPerformanceMeasures(predictions_SVM, LabelsTest, models{i});
case 'Naive Bayes'
[accuracy, sensitivity, specificity] = getPerformanceMeasures(predictions_NB, LabelsTest, models{i});
end

% Add the results to the table
resultsTable(i,:) = {models{i}, accuracy, sensitivity, specificity};
end

% Display the results table
disp(resultsTable);
```

Figure 33 - for loop for results table

The code then iterates over the different models in the 'models' array. For each model, it calculates the performance measures using the 'getPerformanceMeasures' function and the predictions made by the model on the test data. The performance measures are then added to the table as a new row using the 'resultsTable(i,:)' syntax.

Model	Accuracy	Sensitivity	Specificity
"DiagLinear"	0.5000	1	0
"KNN"	0.5000	1	0
"Decision Tree"	0.5000	0	1
"Logistic"	0.8333	1	0.6667
"SVM"	0.8333	1	0.6667
"Naive Bayes"	1	1	1

Finally, the results table is displayed using the *'disp'* function.

and the table:

6x4 table

	1 Model	2 Accuracy	3 Sensitivity	4 Specificity	5	6	7
1	"DiagLinear"	0.5000	1	0			
2	"KNN"	0.5000	1	0			
3	"Decision Tree"	0.5000	0	1			
4	"Logistic"	0.8333	1	0.6667			
5	"SVM"	0.8333	1	0.6667			
6	"Naive Bayes"	1	1	1			
7							

Figure 34 - Performance metrics table for models

Command window output:

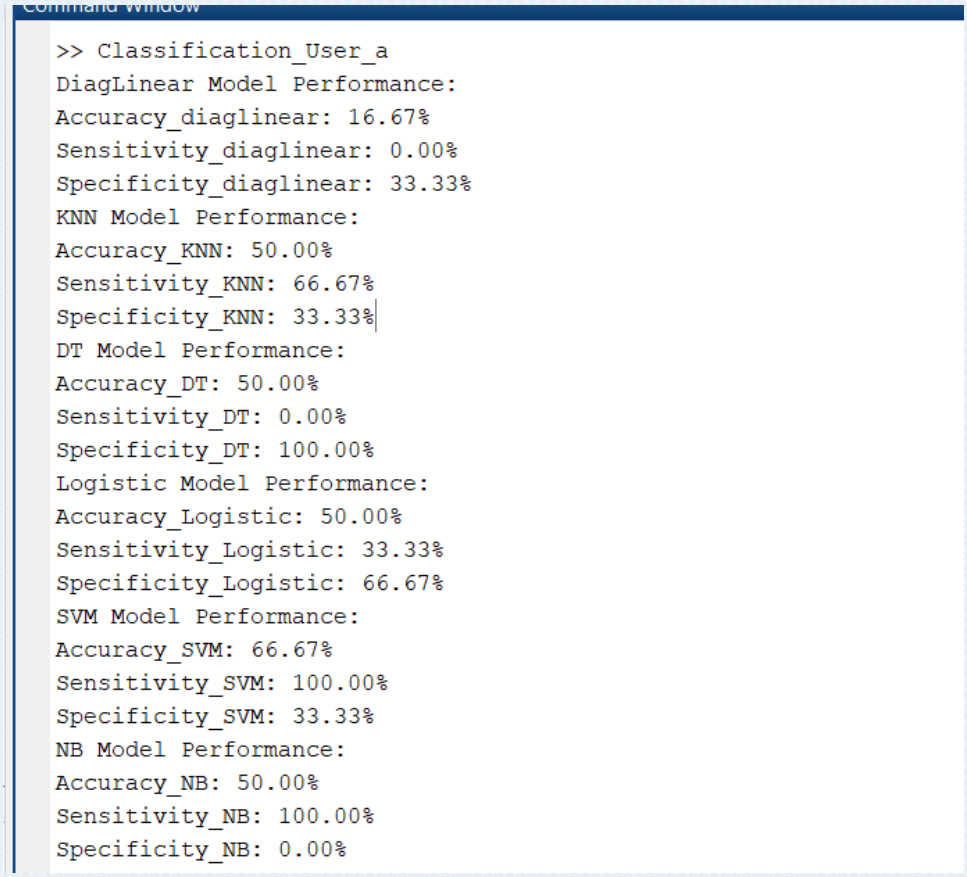


Figure 35 - How performance metrics appear on command window

Model	Accuracy	Sensitivity	Specificity
"DiagLinear"	0.5	1	0
"KNN"	0.5	1	0
"Decision Tree"	0.5	0	1
"Logistic"	0.83333	1	0.66667
"SVM"	0.83333	1	0.66667
"Naive Bayes"	0.66667	1	0.33333

Figure 36 - How performance metrics table appear on command window

Figures

Before filters

After filter

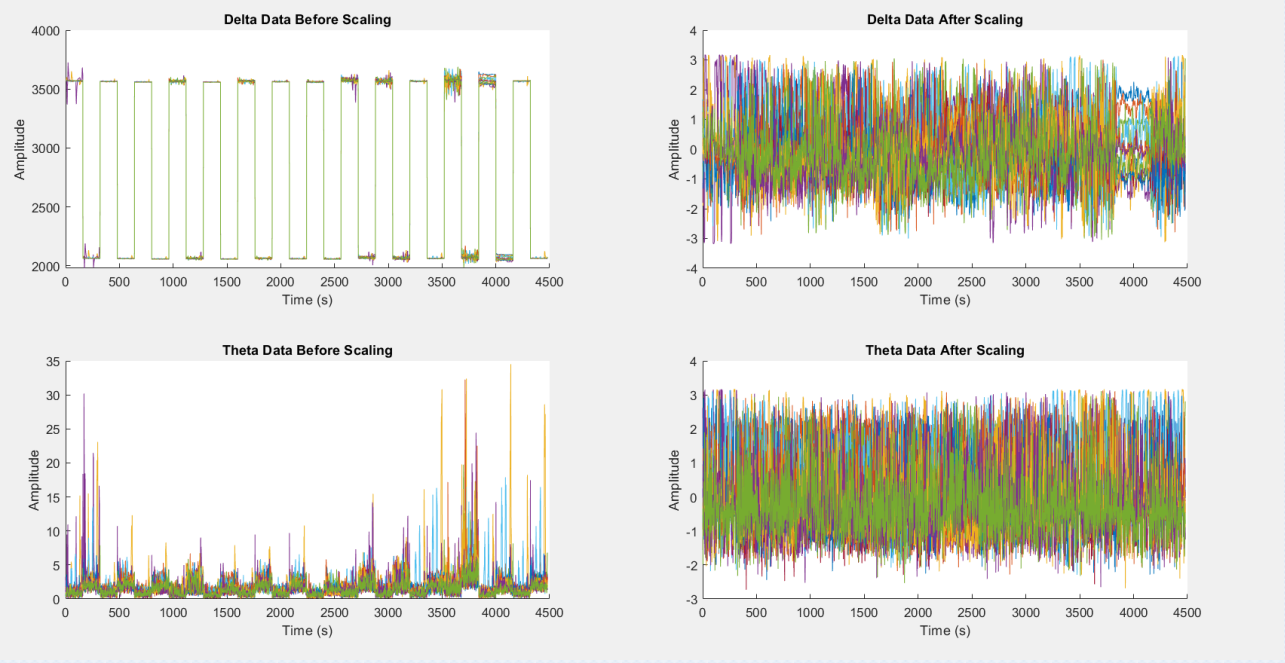


Figure 37- Plotting the whole data before and after scaling

I had to normalize the data, because as you can see the delta data were very big and could have caused bias in the models.

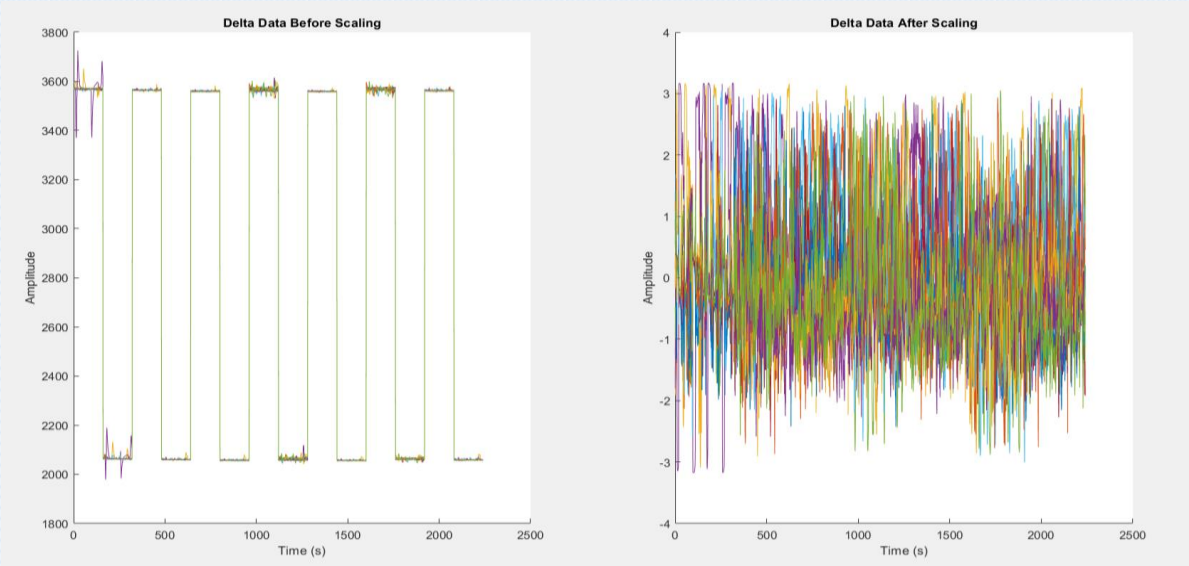


Figure 38 - Plotting the data before and after scaling for 6 Seconds

Different classification setting (IMPROVEMENT)

I decreased the sampling rate down to 20Hz which is 160/8 and makes the data 96 Seconds, and that is valid for our data and actually gives better training setting, as the data itself is bigger which gives the model a chance to learn better.

This code uses in the data splitting Cross validation with nfolds=4 , and that is now usable because the data got bigger, and the iterations of the models over the nfolds gives better accuracy.%

Model	Accuracy	Sensitivity	Specificity
"DiagLinear"	0.70833	0.91667	0.5
"KNN"	0.79167	1	0.58333
"Decision Tree"	0.625	0.5	0.75
"Logistic"	0.70833	0.83333	0.58333
"SVM"	0.75	0.83333	0.66667
"Naive Bayes"	0.54167	0.66667	0.41667

Figure 39 - Accuracy on the 20Hz sampling rate

Discussion

The fluctuation on the accuracy of the models on the 160Hz ranges from 16.6% to 83.33%, and this is considered acceptable because the data is somehow small.

On the other hand, when we decreased the sampling rate, the fluctuation became less obvious as the accuracy range was from 50% to 70.8%, This could be due to the fact that decreasing the sampling rate reduces the amount of data that needs to be processed by the models, making it easier for them to learn and make predictions.

However, it is important to note that decreasing the sampling rate also results in a loss of information, as some of the original data points are discarded. This trade-off between accuracy and information loss should be considered when deciding on the appropriate sampling rate for a given application.

Overall, the results suggest that the decision tree and KNN models performed relatively well on the data, with the highest accuracy rates of around 70%. The other models, such as linear discriminant analysis and logistic regression, had lower accuracy rates but still performed reasonably well. It is worth exploring these models further to see if their performance can be improved through further tuning of the parameters or by using different feature extraction techniques.

In conclusion, the results of this analysis demonstrate the potential of using machine learning models for classifying EEG data which can be used for many projects on the future to enhance human lives.